

Day 10 – Final Integration & Deployment

Goals: Containerise the application and deploy to a cloud provider.

Dockerfile:

```
FROM python:3.12-slim
```

```
# Install uv
```

```
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN uv sync --no-dev
```

```
CMD ["uv", "run", "python", "manage.py", "runserver", "0.0.0.0:8000"]
```

docker-compose.yml:

```
version: '3.9'
```

```
services:
```

```
  web:
```

```
    build: .
```

```
    ports:
```

```
      - "8000:8000"
```

```
    env_file:
```

```
      - .env
```

```
    depends_on:
```

```
      - redis
```

```
    command: uv run python manage.py runserver 0.0.0.0:8000
```

```
  worker:
```

```
    build: .
```

```
    env_file:
```

```
      - .env
```

```
    depends_on:
```

```
      - redis
```

```
    command: uv run celery -A gamekey_platform worker --loglevel=info
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - "6379:6379"
```

```
docker compose up --build
```

Deploying to Render:

1. Push repo to GitHub
2. Create a new **Web Service** on render.com, pointing at your repo

3. Set build command: `uv sync --no-dev && python manage.py migrate`
4. Set start command: `python manage.py runserver 0.0.0.0:$PORT`
5. Add a **Redis** instance (Render provides one) and set `CELERY_BROKER_URL` in environment variables
6. Add a **Background Worker** service using command: `celery -A gamekey_platform worker --loglevel=info`

Final checklist:

- ☐ All endpoints return correct status codes
- ☐ Token auth blocks unauthenticated writes
- ☐ `check_expired_keys` triggers Celery tasks
- ☐ `WebhookDeliveryLog` entries are created per attempt
- ☐ pytest suite passes with $\geq 70\%$ coverage
- ☐ Docker Compose brings up all three services cleanly

Instructor Teaching Plan

Time Breakdown (90 min)

Segment	Duration	Activity
Docker concepts	20 min	Containers vs VMs. Image = blueprint, container = running instance. Layers (each <code>RUN</code> / <code>COPY</code> = one layer, cached independently). Why Docker: "works on my machine" problem solved. Multi-service apps need <code>docker compose</code> .
Write Dockerfile	15 min	Walk through each instruction. Why <code>python:3.12-slim</code> over full Python? Why <code>WORKDIR /app</code> before <code>COPY</code> ? Why <code>uv sync --no-dev</code> (no dev dependencies in production)?
Write <code>docker-compose.yml</code>	15 min	Three services: <code>web</code> , <code>worker</code> , <code>redis</code> . <code>depends_on</code> for startup ordering. <code>env_file</code> for environment variables. Separate command per service — both <code>web</code> and <code>worker</code> use the same image but different entry points.
<code>docker compose up --build</code>	10 min	Build and run. Test <code>POST /api/orders/</code> through the containerized stack. If something fails, read the logs: <code>docker compose logs web</code> .
Render deployment walkthrough	20 min	Push to GitHub. Create Web Service, Worker Service, Redis. Set environment variables (especially <code>CELERY_BROKER_URL</code>). Trigger a deploy.
Final checklist + wrap-up	10 min	Go through the checklist together. Celebrate what students built. Preview: what would the production hardening look like? (Gunicorn, PostgreSQL, <code>ALLOWED_HOSTS</code> , HTTPS.)

Key Concepts to Introduce

- **Docker image vs container** — image is immutable, like a class definition. Container is a running instance, like an object. Multiple containers can run from the same image.
- **CMD vs RUN** — `RUN` executes during build (creates image layers). `CMD` runs when a container starts. `docker-compose.yml` `command`: overrides `CMD`.
- **0.0.0.0 binding** — Django's dev server listens on `127.0.0.1` (loopback) by default. Inside a container, `127.0.0.1` is the container itself — the host machine can't reach it.

0.0.0.0:8000 makes it accessible from outside the container.

- **env_file in compose** — loads the `.env` file into the container's environment. Same as setting each variable manually in the `environment:` section but cleaner.
- **depends_on** — controls start order, not readiness. Redis might not be fully ready when the web service starts. In production, use health checks (`healthcheck:`) or retry logic.
- **Gunicorn vs runserver** — `runserver` is single-threaded, for development only. In production, Gunicorn (or uWSGI) runs multiple worker processes. Render's expected `gunicorn gamekey_platform.wsgi` command vs `runserver`.

⚠ Common Mistakes to Address

- **runserver in production** — Django explicitly warns against this. On Render, use Gunicorn: `gunicorn gamekey_platform.wsgi:application`. Add it with `uv add gunicorn`.
- **ALLOWED_HOSTS not updated** — Django rejects requests from unknown hosts in production. Must include the Render domain: `ALLOWED_HOSTS = ['your-app.onrender.com']` or `['*']` temporarily.
- **SQLite in production** — SQLite is a local file. On Render, the filesystem is ephemeral — the DB is wiped on every redeploy. Use Supabase or Neon for PostgreSQL in production.
- **Not running migrate in the build command** — deploy succeeds but the app crashes immediately on the first DB request. Build command must be: `uv sync --no-dev && python manage.py migrate`.
- **CELERY_BROKER_URL pointing to localhost** — inside Docker Compose, services are accessed by service name, not localhost. `redis://redis:6379/0` (service name `redis`), not `redis://localhost:6379/0`.
- **Worker service missing** — students deploy only the web service. Webhooks are never delivered because no Celery worker is running.

? Check for Understanding

"Why do the `web` and `worker` services use the same Docker image but different `command` values?"

Answer: Both services run the same codebase and need the same environment variables, libraries, and settings. However, the `web` service runs the HTTP web server (`manage.py runserver` or Gunicorn) to handle API requests, while the `worker` service runs the Celery daemon (`celery worker`) to process background tasks. Using the same image simplifies building and keeps the dependencies identical.

"What's the problem with binding the Django dev server to `127.0.0.1` inside a Docker container?"

Answer: `127.0.0.1` is the loopback interface, which is only accessible within the container itself. If the dev server is bound to `127.0.0.1`, the host machine and outer network cannot access the container's port. Binding to `0.0.0.0` tells the container to listen on all interfaces, allowing traffic from the host machine to reach the server.

"Why can't we use SQLite in production on Render?"

Answer: SQLite stores data in a local file on the container's disk. Render containers have an ephemeral filesystem, meaning their disks are wiped and recreated on every deployment, restart, or scale event, resulting in complete database loss. Production environments require a persistent, remote database service (like PostgreSQL).

"What would happen if we forgot to add `python manage.py migrate` to the build command?"

Answer: The application would start, but any database queries (such as listing games, user registration, or purchasing keys) would fail with database errors (e.g., `Relation does not exist` or `no such table`) because the database tables would not match the new code models.

✓ Day Checkpoint (Final)

`docker compose up --build` starts all three services without errors. `POST /api/orders/` returns 201 through the containerized stack. The Celery worker (in the worker container) logs task execution. If deployed to Render: the web service URL returns a valid API response and the worker service shows as "running" in the Render dashboard. `pytest` suite passes with $\geq 70\%$ coverage.

Project Structure (End of Day 10)

```
gamekey_platform/
├── gamekey_platform/
│   ├── __init__.py          # Loads Celery app
│   ├── celery.py
│   ├── settings.py
│   └── urls.py
├── games/
│   ├── management/
│   │   └── commands/
│   │       └── check_expired_keys.py
│   ├── migrations/
│   ├── tests/
│   │   └── test_webhook.py
│   ├── admin.py
│   ├── models.py           # Publisher, Game, GameKey, Order, OrderIt
│   ├── permissions.py
│   ├── serializers.py
│   ├── tasks.py            # Celery async webhook task
│   ├── viewsets.py
│   ├── views.py            # register, create_order
│   └── webhooks.py         # sync helper (Day 7 reference)
├── .env
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── manage.py
└── pytest.ini
```

API Reference — Sample Requests & Responses

All endpoints are prefixed with `/api/`. Authenticated endpoints require the header:

`Authorization: Token <your-token>`

POST `/api/register/`

Register a new user and receive an auth token.

Request

```
POST /api/register/  
Content-Type: application/json
```

```
{  
  "username": "dheeresh",  
  "password": "securepass123"  
}
```

Response 201 Created

```
{  
  "token": "9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a"  
}
```

Response 400 Bad Request — missing fields

```
{  
  "error": "Username and password required."  
}
```

GET `/api/publishers/`

List all publishers. Public endpoint (no auth needed).

Request

```
GET /api/publishers/
```

Response 200 OK

```
[  
  {  
    "id": 1,  
    "name": "Valve Corporation",  
    "webhook_url": "https://valve.example.com/webhooks/gamekey",  
    "user": 2  
  }  
]
```

```
    },
    {
      "id": 2,
      "name": "CD Projekt Red",
      "webhook_url": "https://cdpr.example.com/hooks/expiry",
      "user": 3
    }
  ]
```

webhook_secret is always excluded from responses.

POST /api/publishers/

Create a publisher profile. Requires auth.

Request

```
POST /api/publishers/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json
```

```
{
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "webhook_secret": "mysecret_abc123",
  "user": 2
}
```

Response 201 Created

```
{
  "id": 1,
  "name": "Valve Corporation",
  "webhook_url": "https://valve.example.com/webhooks/gamekey",
  "user": 2
}
```

GET /api/games/

List all games. Public endpoint.

Request

```
GET /api/games/
```

Response 200 OK

```
[
  {
    "id": 1,
    "title": "Half-Life 3",
```

```
    "publisher": 1,
    "price": "29.99"
  },
  {
    "id": 2,
    "title": "Cyberpunk 2077",
    "publisher": 2,
    "price": "49.99"
  }
]
```

POST /api/games/

Create a game. Requires auth.

Request

```
POST /api/games/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json
```

```
{
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

Response 201 Created

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

GET /api/games/{id}/

Retrieve a single game.

Request

```
GET /api/games/1/
```

Response 200 OK

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "29.99"
}
```

```
}
```

Response 404 Not Found

```
{
  "detail": "No Game matches the given query."
}
```

PATCH /api/games/{id}/

Partially update a game. Requires auth + must be the publisher's owner.

Request

```
PATCH /api/games/1/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json
```

```
{
  "price": "19.99"
}
```

Response 200 OK

```
{
  "id": 1,
  "title": "Half-Life 3",
  "publisher": 1,
  "price": "19.99"
}
```

Response 403 Forbidden — wrong owner

```
{
  "detail": "You do not have permission to perform this action."
}
```

POST /api/orders/

Buy a game key. Requires auth. Atomically assigns or generates a key with a 30-day expiry.

Request

```
POST /api/orders/
Authorization: Token 9a4f2c8b3e1d6f0a7c5b2e8d4f1a3c7e9b5d2f0a
Content-Type: application/json
```

```
{
  "game_id": 1
}
```

Response 201 Created

```
{
  "order_id": 42,
  "game": "Half-Life 3",
  "key": "A3F9-B21C-4DE7-9901-CC84",
  "expires_at": "2026-07-16T10:30:00Z"
}
```

Response 400 Bad Request — missing game_id

```
{
  "error": "game_id is required."
}
```

Response 404 Not Found — game doesn't exist

```
{
  "error": "Game not found."
}
```

Response 401 Unauthorized — no token provided

```
{
  "detail": "Authentication credentials were not provided."
}
```

Webhook Payload — game_key.expired

Sent asynchronously by Celery to the publisher's `webhook_url` when a key expires. Signed with HMAC-SHA256.

Headers

Content-Type: application/json
X-Signature: sha256=3c6e9b52a3c2ac3f7dca0c944b6d68b3f15ea2e3c0cf1b2e5f7

Body

```
{
  "event": "game_key.expired",
  "game_key": "A3F9-B21C-4DE7-9901-CC84",
  "game_title": "Half-Life 3",
  "expired_at": "2026-07-16T10:30:00Z",
  "attempt": 0
}
```

attempt increments on each retry (max 3). Verify the signature on your end:

```
import hmac, hashlib, json
expected = hmac.new(secret.encode(), json.dumps(payload, sort_keys=
assert hmac.compare_digest(f"sha256={expected}", request.headers['
```

Webhook Retry Behaviour

Attempt	Delay before retry
0 → 1	60 seconds
1 → 2	120 seconds
2 → 3	240 seconds
3	Task marked failed, logged

All attempts (success or failure) are recorded in `WebhookDeliveryLog`.

Error Reference

Status	Meaning
200 OK	Successful read
201 Created	Resource created
400 Bad Request	Missing or invalid input
401 Unauthorized	No or invalid token
403 Forbidden	Authenticated but not the owner
404 Not Found	Resource does not exist

10 days · 1.5 hours/day · production-ready Django API with async webhooks